Product Requirements Document (PRD)

Cookidoo LLM Assistant

Version: 1.0
Date: November 28, 2025
Status: Draft

---

1. Executive Summary

The Cookidoo LLM Assistant represents a transformative approach to home cooking by bridging the gap between Cookidoo's extensive recipe library and the personalized needs of modern home cooks. This AI-powered platform leverages cutting-edge large language models to provide an intelligent layer on top of the Cookidoo ecosystem, enabling capabilities that are currently unavailable in the native platform.

At its core, this solution addresses a fundamental limitation: while Cookidoo offers thousands of curated recipes, users cannot easily customize these recipes to match their dietary preferences, family sizes, or ingredient availability. The Cookidoo LLM Assistant acts as an intelligent intermediary, understanding recipe structures, ingredient relationships, and cooking techniques well enough to suggest meaningful modifications while maintaining the integrity and quality of the original recipes.

By integrating with Databricks and Azure's cloud infrastructure, we're not just building a tool for today—we're creating a data-driven platform that learns from user interactions, continuously improves its recommendations, and showcases the power of modern AI in everyday life. This project demonstrates how enterprise-grade AI platforms can be applied to consumer-facing applications, making sophisticated technology accessible and useful for home cooks worldwide.

---

2. Objective

Our primary objective is to create an intelligent cooking companion that fundamentally changes how users interact with Cookidoo recipes. Rather than viewing recipes as static instructions, we envision a dynamic, conversational experience where users can explore variations, ask questions, and adapt recipes to their unique situations—all while maintaining the quality and reliability that Cookidoo is known for.

The platform will achieve this through several interconnected goals:

- **Enable Recipe Customization**: Users should be able to modify any Cookidoo recipe with natural language requests like "make this vegan" or "reduce the sugar by half," with the AI understanding not just the literal request but the cascading implications for cooking times, temperatures, and complementary ingredients.
- **Provide Intelligent Assistance**: Beyond simple recipe retrieval, the assistant must understand cooking principles deeply enough to answer questions like "can I substitute buttermilk with yogurt?" or "why did my sauce separate?" This requires domain expertise encoded in the model.
- **Automate Meal Planning**: By analyzing multiple recipes together, the system should consolidate shopping lists, identify ingredient overlaps, and suggest complementary recipes based on what users already plan to cook.
- **Enable Continuous Learning**: Every interaction provides data that can improve the system. By storing interactions in Databricks Delta Live Tables, we create a feedback loop where the model becomes more accurate, more helpful, and better aligned with user needs over time.
- **Deliver Seamless Integration**: The experience should feel native and natural, whether accessed through a web interface, mobile app, or programmatic API, ensuring that the technology enhances rather than complicates the cooking experience.

---

3. Problem Statement

The modern home cook faces a paradox: while platforms like Cookidoo provide access to thousands of professionally developed recipes, the rigid, one-size-fits-all nature of these recipes often creates more friction than convenience. This disconnect between the promise of digital cooking platforms and the reality of individual cooking needs represents a significant opportunity for innovation.

Current Challenges

Cookidoo's current architecture, while excellent for delivering standardized recipes, fundamentally limits user agency in several critical ways:

1. **Limited Recipe Customization**: The platform treats recipes as immutable documents. Users cannot easily modify ingredient quantities beyond simple serving size adjustments, cannot substitute ingredients intelligently, and cannot adapt cooking methods to their available equipment. This rigidity means a recipe designed for a conventional oven cannot be easily adapted for an air fryer or slow cooker, forcing users to search for entirely different recipes or experiment blindly.
2. **No Recipe Export**: Once users identify recipes they want to modify or save in their own format, they hit a wall. Cookidoo lacks export functionality, trapping recipe data within the platform. This creates particular frustration when users want to maintain their own recipe collections with personal notes and adaptations.
3. **Manual Shopping List Creation**: When planning multiple meals, users must manually aggregate ingredients across recipes, adjust quantities, and organize items by store section.

This tedious process often takes 20-30 minutes for a week's meal planning, time that could be automated entirely.

4. **No Intelligent Assistance**: When users have questions—"What can I use instead of cardamom?" or "How do I know when the onions are properly caramelized?"—they must leave the platform to search elsewhere. The absence of contextual, intelligent assistance means users constantly context-switch between Cookidoo and search engines or cooking forums.
5. **Limited Recipe Discovery**: Current search and filter mechanisms rely on exact tag matches. Users with complex requirements—"high-protein, low-carb, uses chicken thighs, under 30 minutes"—must manually filter and cross-reference, often missing perfectly suitable recipes that don't match exact search terms.

User Pain Points

These technical limitations translate into daily frustrations that erode the value proposition of the Cookidoo platform:

- **Dietary Restrictions**: Users with allergies, intolerances, or dietary preferences (vegan, keto, low-sodium) cannot easily adapt the vast majority of recipes, effectively reducing the usable recipe library by 70-80%.
- **Household Variability**: Recipes designed for 4 servings don't always scale linearly. When cooking for 2 or 8, users struggle with ingredient quantities, cooking times, and pan sizes without guidance.
- **Ingredient Availability**: When a recipe calls for an ingredient users don't have (and don't want to buy for one recipe), there's no easy way to understand what makes a good substitute and how it might affect the final dish.
- **Time Investment in Planning**: The manual overhead of meal planning—reviewing recipes, compiling shopping lists, checking pantry inventory—often takes longer than the actual cooking, discouraging users from planning ahead.
- **Lost Customizations**: When users do experiment successfully with modifications, there's no way to save those insights within Cookidoo, meaning they must recreate the recipe from memory or maintain external notes.

---

4. Target Users

Understanding our users is critical to building a product that genuinely solves real problems rather than adding complexity. Our user base spans a spectrum of cooking experience and technical sophistication, but they share common needs: flexibility, intelligence, and efficiency in their cooking workflow.

Primary Users

Home Cooks (The Pragmatists) These are individuals who use Cookidoo regularly—typically 3-5 times per week—as their primary source of dinner inspiration. They're comfortable with technology but not necessarily tech-savvy. They value reliability and simplicity over advanced features. Their typical use case involves browsing recipes after work, selecting something that fits their available time and ingredients, and following instructions closely. They need the LLM assistant to be intuitive and trustworthy, providing modifications that reliably work without requiring deep cooking knowledge to evaluate.

Meal Planners (The Organizers) This segment plans their weekly or bi-weekly meals in advance, often on weekends. They're optimization-focused, looking to minimize grocery shopping trips, reduce food waste, and ensure variety in their family's diet. They frequently cook for families of 4-6 people and are highly motivated by time-saving features. For them, the shopping list consolidation and multi-recipe analysis features are paramount. They're willing to invest time upfront in planning if it saves time during the week.

Dietary-Restricted Users (The Adapters) This increasingly important segment includes people managing celiac disease, lactose intolerance, severe allergies, or following specific diets like vegan, keto, or low-FODMAP. They currently face the most friction with standard recipe platforms, often needing to mentally translate recipes or skip them entirely. They're highly motivated users who will engage deeply with customization features because the alternative—finding suitable recipes without modification—is so time-consuming. They need the AI to understand not just ingredient swaps but the underlying principles of their dietary restrictions.

Secondary Users

Cooking Enthusiasts (The Experimenters) These users view cooking as a hobby and creative outlet. They're less interested in following recipes exactly and more interested in understanding principles and trying variations. They'll push the boundaries of the system, asking complex questions and requesting unusual modifications. While smaller in number, they're valuable for testing the limits of the AI and providing detailed feedback. They're also likely to be advocates, sharing their experiences and creative uses on social media and cooking forums.

Busy Professionals (The Efficiency Seekers) Time-poor but often willing to invest in tools that save time, these users want cooking to be as frictionless as possible. They're likely to use the API integration features, potentially incorporating the assistant into their existing productivity workflows. They value speed and reliability over customization depth. Quick answers to substitution questions or rapid shopping list generation are their primary use cases. They're most likely to churn if the system adds complexity rather than removing it.

---

5. Key Features & Requirements

5.1 Core Functionality

The platform's core functionality centers on three interconnected capabilities that work together to create a seamless cooking experience. Each feature is designed not just to solve an individual problem but to contribute to an overall workflow that reduces friction at every step of meal preparation.

Recipe Modification & Generation

At the heart of our platform is the ability to intelligently transform recipes while maintaining their essential character and reliability. This goes far beyond simple find-and-replace operations on ingredients.

When a user requests a modification—"make this dairy-free" or "I only have a slow cooker"—the system must understand the cascading implications. Removing dairy from a cake isn't just about swapping milk for almond milk; it affects moisture content, baking time, potentially pH balance, and might require compensating with additional fat or binding agents. The LLM must reason through these relationships, drawing on deep culinary knowledge to produce modifications that actually work.

Key capabilities include:

- **Accept existing Cookidoo recipes as input** with full parsing of ingredients, quantities, instructions, and metadata
- **Process user modification requests** spanning ingredients, quantities, cooking methods, equipment substitutions, and dietary adaptations
- **Generate new recipe variations** that maintain structural integrity and culinary principles while meeting user requirements
- **Save modified recipes to "My Cookidoo Recipes"** seamlessly through MCP server integration, making personalized recipes feel like native Cookidoo content
- **Maintain recipe structure and formatting consistency** so modified recipes are indistinguishable in quality from original Cookidoo recipes

Intelligent Q&A

Cooking is inherently uncertain, filled with questions that arise in the moment: "Is this the right consistency?" "Can I prep this ahead?" "What if I don't have that spice?" Traditional recipe platforms leave users to figure these out alone or interrupt their workflow to search elsewhere.

Our Q&A system acts as an always-available cooking mentor, providing contextual answers grounded in the specific recipe the user is working with. When asked about substitutions, it considers not just ingredient equivalency but how the substitution affects the recipe's technique, timing, and final result.

Key capabilities include:

- **Answer questions about recipes, ingredients, and cooking techniques** with context awareness of the user's current recipe and cooking history

- **Provide substitution recommendations** that consider flavor profiles, cooking properties, and dietary implications
- **Offer cooking tips and best practices** tailored to the user's skill level and previous interactions
- **Explain cooking terminology and methods** in accessible language, building user confidence and capability over time

Shopping List Management

Meal planning should reduce mental overhead, not increase it. Yet the manual process of reviewing multiple recipes, extracting ingredients, consolidating quantities, and organizing by store section often takes 30+ minutes and is prone to errors.

Our shopping list system automates this entirely, but with intelligence that goes beyond simple aggregation. It recognizes when "1 onion" in one recipe and "1 medium onion, diced" in another refer to the same item. It understands that if you're buying fresh basil for one recipe, other recipes calling for dried basil could be enhanced by using fresh instead. It can suggest buying in bulk when multiple recipes use the same ingredient.

Key capabilities include:

- **Generate consolidated shopping lists from selected recipes** with intelligent ingredient matching and quantity aggregation
- **Organize ingredients by category** (produce, dairy, pantry, etc.) and optionally by store layout for specific retailers
- **Calculate quantities for multiple recipes** with smart rounding ("buy 1 lb of ground beef" not "0.94 lbs")
- **Support serving size adjustments** that propagate through all recipes and update the shopping list dynamically

5.2 Integration Requirements

Cookidoo API Integration

- Leverage open-source API clients for Cookidoo platform interaction
- Implement MCP (Model Context Protocol) server for standardized API access
- Ensure reliable recipe retrieval and saving functionality
- Handle authentication and session management

Databricks Integration

- Store recipe data in Delta Live Tables (DLT)
- Track user interactions and agent performance metrics
- Enable data analysis and visualization through Databricks notebooks
- Support continuous model improvement through interaction analysis

AgentBricks Integration

- Manage LLM agent workflows and orchestration
- Handle complex multi-step recipe generation tasks
- Coordinate interactions between Cookidoo API and LLM
- Enable workflow monitoring and debugging

5.3 Security & Compliance

Credential Management

- Use Azure Key Vault for secure credential storage
- Implement token-based authentication for API access
- Never expose credentials in codebase or logs
- Support credential rotation and expiration handling

Data Privacy

- Ensure user data protection and GDPR compliance
- Implement secure data transmission (HTTPS/TLS)
- Provide data retention and deletion policies
- Maintain audit logs for security events

5.4 Observability & Monitoring

Logging

- Implement structured logging for all agent activities
- Track API calls, errors, and performance metrics
- Store logs in centralized system for analysis
- Support log-based alerting and monitoring

Error Handling

- Graceful error handling with user-friendly messages
- Retry logic for transient API failures
- Fallback mechanisms for service degradation
- Error tracking and reporting

---

6. Technical Architecture

6.1 High-Level Components

Frontend

- **Technology**: React-based web application
- **Features**: Recipe browser, modification interface, shopping list viewer
- **Deployment**: Containerized for Azure Container Apps

Backend

- **Technology**: FastAPI or Flask
- **Features**: API endpoints, LLM orchestration, business logic
- **Deployment**: Dockerized microservices on Azure Container Apps

LLM Agent

- **Technology**: AgentBricks with state-of-the-art LLM
- **Features**: Recipe understanding, modification, generation
- **Training**: Custom datasets for cooking domain expertise

Data Layer

- **Technology**: Databricks Delta Live Tables
- **Features**: Recipe storage, interaction tracking, analytics
- **Integration**: Databricks notebooks for data exploration

MCP Server

- **Purpose**: Standardized Cookidoo API integration
- **Features**: Recipe CRUD operations, authentication management
- **Deployment**: Standalone service or Databricks-hosted

6.2 Integration Points

```
User Interface (React)
    ↓
Backend API (FastAPI/Flask)
    ↓
AgentBricks LLM Orchestration
    ↓
├→ MCP Server → Cookidoo API
├→ Databricks DLT (Data Storage)
└→ Azure Key Vault (Credentials)
```

---

7. User Stories

Epic 1: Recipe Modification

- **US1.1**: As a user, I want to modify recipe ingredients so I can adapt recipes to my dietary needs
- **US1.2**: As a user, I want to adjust serving sizes so I can cook for different group sizes
- **US1.3**: As a user, I want to save modified recipes so I can access them later

Epic 2: Intelligent Assistance

- **US2.1**: As a user, I want to ask questions about ingredients so I can make informed substitutions
- **US2.2**: As a user, I want cooking technique explanations so I can improve my skills
- **US2.3**: As a user, I want recipe recommendations based on my preferences

Epic 3: Shopping List

- **US3.1**: As a user, I want to generate shopping lists from multiple recipes so I can shop efficiently
- **US3.2**: As a user, I want organized ingredient lists so I can navigate the store easily
- **US3.3**: As a user, I want to adjust quantities so I can plan for different meal counts

---

8. Success Metrics

User Engagement

- Number of active users (DAU/MAU)
- Recipe modifications per user
- Average session duration
- Feature adoption rates

Technical Performance

- API response time (p95 < 2 seconds)
- System uptime (99.9% SLA)
- Error rate (< 1%)
- LLM response quality (user satisfaction > 4.5/5)

Business Impact

- User retention rate
- Recipe save conversion rate

- Shopping list generation usage
- User satisfaction scores (NPS)

---

9. Technical Specifications

9.1 Development Standards

- **Code Style**: PEP 8 for Python
- **Documentation**: Comprehensive docstrings and README files
- **Testing**: Unit tests with >80% coverage
- **Version Control**: Git with feature branch workflow

9.2 Infrastructure Requirements

- **Container Platform**: Docker for all services
- **Cloud Provider**: Microsoft Azure
- **Compute**: Azure Container Apps
- **Storage**: Databricks Delta Lake
- **Security**: Azure Key Vault

9.3 API Design

- RESTful API endpoints
- JSON request/response format
- API versioning strategy
- Rate limiting and authentication
- Comprehensive API documentation (OpenAPI/Swagger)

---

10. Deployment Strategy

Phase 1: MVP (Minimum Viable Product)

- Basic recipe modification functionality
- Simple Q&A capability
- Shopping list generation
- Web UI with core features
- Databricks integration for data storage

Phase 2: Enhanced Features

- Advanced LLM training with cooking datasets
- MCP server deployment
- Enhanced error handling and logging
- Performance optimization
- Analytics dashboards

Phase 3: Production Release

- Full security hardening
- Comprehensive monitoring and alerting
- Load testing and scalability improvements
- Complete documentation
- Production deployment to Azure

---

11. Risks & Mitigation

Risk	Impact	Probability	Mitigation
Cookidoo API changes	High	Medium	Version API client, implement adapter pattern
LLM hallucinations in recipes	High	Medium	Implement validation layer, user review process
Azure service costs	Medium	High	Implement cost monitoring, optimize resource usage
Security vulnerabilities	High	Low	Regular security audits, dependency scanning
Poor LLM performance	Medium	Medium	Continuous training, user feedback loop

---

12. Dependencies

External Dependencies

- Cookidoo API availability and stability
- Open-source Cookidoo API client maintenance
- Azure service availability
- Databricks platform reliability
- LLM model availability and performance

Internal Dependencies

- Development team capacity
- Access to Azure and Databricks environments
- Cookidoo account credentials for testing

- Training datasets for LLM fine-tuning

---

13. Timeline (Preliminary)

Phase 1: Foundation (Weeks 1-4)

- Project setup and infrastructure provisioning
- Cookidoo API integration
- Basic LLM agent implementation
- Core backend API development

Phase 2: Feature Development (Weeks 5-8)

- Recipe modification engine
- Q&A functionality
- Shopping list generator
- Frontend development

Phase 3: Integration & Testing (Weeks 9-10)

- Databricks integration
- AgentBricks implementation
- End-to-end testing
- Security hardening

Phase 4: Deployment & Documentation (Weeks 11-12)

- Production deployment
- Documentation completion
- User acceptance testing
- Launch preparation

---

14. LLM Model Selection & Rationale

Selecting the appropriate large language model is perhaps the most critical technical decision for this project, as it directly impacts response quality, cost, latency, and long-term maintainability. We've evaluated five leading options available in the Databricks ecosystem, each with distinct trade-offs.

Model Evaluation Criteria

Before diving into specific models, we established evaluation criteria based on our product requirements:

1. **Cooking Domain Understanding**: Ability to understand recipe structures, ingredient relationships, and cooking techniques
2. **Instruction Following**: Precision in following user modification requests without hallucinating invalid recipes
3. **Response Quality**: Coherence, accuracy, and helpfulness of generated content
4. **Latency**: Time to first token and overall response time (target: <2 seconds p95)
5. **Cost**: Per-token pricing at expected scale (estimated 10M tokens/day at full scale)
6. **Databricks Integration**: Ease of deployment and management within Databricks environment
7. **Context Window**: Ability to handle full recipe contexts plus conversation history
8. **Fine-tuning Capability**: Ability to adapt to cooking domain with custom training data

Model Options Analysis

Option 1: Claude Sonnet 4.5 (Recommended Primary Choice)

Overview: Anthropic's Claude Sonnet 4.5 represents the latest evolution of the Claude family, offering an optimal balance of capability, speed, and cost. It's specifically designed for production applications requiring high-quality reasoning with practical latency constraints.

Strengths:

- **Exceptional Instruction Following**: Claude models excel at following complex, multi-step instructions precisely—critical when users request modifications like "make this vegan and double the recipe"
- **Strong Reasoning**: Demonstrates deep understanding of cause-and-effect relationships in cooking (e.g., why reducing sugar affects texture and browning)
- **Safety and Reliability**: Lower hallucination rates compared to alternatives, crucial for recipe generation where incorrect instructions could ruin meals
- **Context Window**: 200K tokens allows including multiple recipes, full conversation history, and reference materials
- **Databricks Integration**: Native support through Databricks Model Serving with streamlined deployment
- **Cost-Effective**: At ~$3 per million tokens (input) and ~$15 per million tokens (output), provides excellent value for quality
- **Response Quality**: Consistently produces well-structured, readable recipe modifications that maintain the tone and style of original recipes

Weaknesses:

- Limited fine-tuning options compared to open-source alternatives
- Slightly higher cost than base-level alternatives (though offset by quality)
- Dependent on Anthropic's API availability

Use Case Fit: Ideal for user-facing recipe modification and Q&A where quality and reliability are paramount. The model's strong instruction-following and low hallucination rate make it trustworthy for generating recipes users will actually cook.

Recommendation: **Primary model for user-facing interactions**

Option 2: Claude Opus 4.1

Overview: The flagship Claude model offering maximum capability for the most complex reasoning tasks.

Strengths:

- **Highest Capability**: Best-in-class reasoning for complex culinary questions and multi-recipe meal planning
- **Exceptional Context Understanding**: Excels at synthesizing information across multiple recipes to generate consolidated shopping lists
- **Superior Domain Adaptation**: Can handle edge cases and unusual requests that might confuse smaller models
- **Same Integration Benefits**: Full Databricks support with straightforward deployment

Weaknesses:

- **Significantly Higher Cost**: At ~$15 per million input tokens and ~$75 per million output tokens (5x more expensive than Sonnet)
- **Slower Response Times**: Typically 20-30% slower than Sonnet, potentially exceeding latency targets
- **Overkill for Most Tasks**: The additional capability isn't necessary for standard recipe modifications

Use Case Fit: Best reserved for complex analytical tasks like generating meal plans for entire weeks with nutritional balancing, or analyzing large batches of recipes for pattern extraction in our data analytics pipeline.

Recommendation: **Reserve for backend analytical tasks and complex edge cases; not cost-effective for standard user interactions**

Option 3: OpenAI GPT-4 Turbo/GPT-5 (Future Consideration)

Overview: OpenAI's flagship models, with GPT-5 anticipated to launch in 2026, represent the industry standard for general-purpose LLM applications.

Strengths:

- **Broad Capability**: Excellent general knowledge including extensive cooking and recipe information
- **Strong Ecosystem**: Extensive tooling, libraries, and community support
- **Databricks Integration**: Supported through external model serving with some additional setup
- **Function Calling**: Native support for structured outputs and tool use, valuable for MCP server integration
- **GPT-5 Promise**: Next generation expected to offer significant improvements in reasoning and efficiency

Weaknesses:

- **Higher Cost**: GPT-4 Turbo pricing (~$10 per million input tokens, ~$30 per million output) is competitive but GPT-5 pricing unknown
- **Consistency Concerns**: GPT-4 sometimes produces verbose, over-explained responses less suitable for recipe contexts
- **Less Precise Instruction Following**: Compared to Claude, more prone to embellishing or deviating from specific requests
- **Integration Complexity**: Requires external API calls or more complex Databricks setup compared to native Anthropic integration
- **Context Window**: 128K tokens sufficient but smaller than Claude

Use Case Fit: Viable alternative with strong general capabilities, but doesn't offer compelling advantages over Claude Sonnet for our specific use case. GPT-5, when available, should be re-evaluated.

Recommendation: **Monitor GPT-5 development; not recommended for initial deployment given Claude's advantages in instruction-following and Databricks integration**

Option 4: Open Source Models (Llama 3.1/3.2, Mixtral)

Overview: Open-source alternatives like Meta's Llama 3.1 (70B/405B) and Mistral's Mixtral (8x7B) offer deployment flexibility and cost advantages.

Strengths:

- **Cost Control**: Once deployed, only infrastructure costs (no per-token fees), potentially 80-90% cheaper at scale

- **Data Privacy**: Can be deployed entirely within our Azure/Databricks environment with no external API calls
- **Customization**: Full fine-tuning capability with our cooking domain dataset
- **Databricks Optimized**: Can leverage Databricks' optimized inference infrastructure for maximum performance
- **No Vendor Lock-in**: Complete control over model lifecycle and updates

Weaknesses:

- **Quality Gap**: Even Llama 3.1 405B doesn't match Claude Sonnet 4.5 or GPT-4 in reasoning quality and instruction-following
- **Infrastructure Overhead**: Requires significant GPU resources (A100 or H100 clusters) for inference, with associated complexity
- **Latency Challenges**: Larger variants (405B) struggle to meet <2 second latency targets without expensive multi-GPU setups
- **Fine-tuning Required**: Out-of-box performance insufficient; requires substantial fine-tuning investment
- **Ongoing Maintenance**: Model management, scaling, and updates become internal responsibilities

Use Case Fit: Most viable for high-volume, price-sensitive operations once user base reaches scale (>1M daily active users). Requires upfront investment in fine-tuning and infrastructure.

Recommendation: **Plan for migration path once scale justifies infrastructure investment (Year 2+); not recommended for MVP given quality gap and engineering overhead**

Option 5: Specialized Smaller Models (GPT-3.5, Claude Haiku)

Overview: Faster, cheaper alternatives for simple tasks that don't require advanced reasoning.

Strengths:

- **Very Low Cost**: GPT-3.5 (~$0.50 per million tokens) and Claude Haiku (~$0.25 per million input tokens) dramatically reduce costs
- **Low Latency**: Sub-second response times for simple queries
- **Sufficient for Simple Tasks**: Can handle straightforward questions, simple substitutions, and basic recipe reformatting

Weaknesses:

- **Limited Reasoning**: Struggle with complex modifications or multi-step instructions
- **Higher Error Rates**: More prone to mistakes in recipe generation that could frustrate users
- **Poor Edge Case Handling**: Fail on unusual requests or complex dietary restrictions

- **Degraded User Experience**: Quality difference noticeable to users, particularly for core features

Use Case Fit: Could serve as a routing layer for triaging simple vs. complex requests, or for non-critical features like FAQ responses.

Recommendation: **Use as a cost-optimization strategy for specific, well-defined simple tasks; not suitable as primary model**

Final Recommendation: Hybrid Architecture

Based on the analysis above, we recommend a **hybrid approach** that optimizes for quality, cost, and scalability:

Phase 1 (MVP - Months 1-6)

- **Primary Model**: Claude Sonnet 4.5 for all user-facing interactions
- **Analytics Model**: Claude Opus 4.1 for complex batch analysis in Databricks notebooks
- **Estimated Cost**: $3,000-5,000/month at 100K users with 10 interactions/user/month

Phase 2 (Scale - Months 7-18)

- **Primary Model**: Claude Sonnet 4.5 for complex recipe modifications and Q&A
- **Secondary Model**: Claude Haiku for simple queries (routing layer)
- **Analytics Model**: Fine-tuned Llama 3.1 70B for cost-effective batch processing
- **Estimated Cost**: $15,000-25,000/month at 500K users

Phase 3 (Mature - Month 18+)

- **Primary Model**: Fine-tuned Llama 3.2 405B for core features (self-hosted)
- **Fallback Model**: Claude Sonnet 4.5 for edge cases and quality assurance
- **Analytics Model**: Specialized fine-tuned model for Databricks analytics
- **Estimated Cost**: $30,000-40,000/month infrastructure + $5,000/month API costs at 2M+ users

Implementation in Databricks

Immediate Setup:

```
# Databricks Model Serving configuration for Claude Sonnet 4.5
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import ServedModelInput
```

```
w = WorkspaceClient()
w.serving_endpoints.create(
    name="cookidoo-llm-primary",
    config={
        "served_models": [{
            "name": "claude-sonnet-4-5",
            "model_name": "anthropic.claude-sonnet-4-5",
            "workload_size": "Small",
            "scale_to_zero_enabled": True
        }]
    }
)
```

Monitoring & Optimization:

- Track cost per interaction, response quality scores, and latency
- Implement A/B testing framework to evaluate model alternatives
- Build routing logic to direct queries to appropriate model tier
- Establish quality thresholds for automatic fallback to more capable models

Conclusion

Claude Sonnet 4.5 offers the best combination of quality, cost, latency, and Databricks integration for our initial deployment. Its exceptional instruction-following and low hallucination rate make it ideal for recipe generation where errors have real-world consequences. The hybrid architecture provides a clear path to cost optimization as we scale while maintaining the quality standards users expect.

---

15. Open Questions

Beyond the model selection analysis above, several strategic and operational questions require stakeholder input:

1. What is the expected user load and scalability requirements for launch vs. year 1 vs. year 2?
2. Are there specific Cookidoo API rate limits or terms of service restrictions we need to consider?
3. What is the approved budget for Azure and Databricks resources across development, staging, and production environments?
4. Are there specific compliance requirements (GDPR, CCPA, HIPAA for nutrition data) that affect architecture decisions?
5. What is the target geography for initial launch, and does this affect data residency requirements?
6. What level of recipe validation and safety checking is required before saving modifications to user accounts?

7. Do we need human-in-the-loop review for certain types of modifications (allergen changes, major ingredient substitutions)?
8. What's our strategy if Cookidoo changes their API or terms of service?
9. How do we handle liability if AI-generated recipe modifications lead to food safety issues?
10. What's the monetization strategy—subscription, freemium, usage-based pricing?

---

16. Appendices

Appendix A: Technical Stack Summary

- **Frontend**: React
- **Backend**: FastAPI/Flask
- **LLM Framework**: AgentBricks
- **Data Platform**: Databricks (Delta Live Tables)
- **Cloud Provider**: Microsoft Azure
- **Container Platform**: Docker
- **Secrets Management**: Azure Key Vault
- **API Integration**: MCP Server

Appendix B: Related Documents

- Technical Architecture Document (TBD)
- API Specification Document (TBD)
- Deployment Guide (TBD)
- User Guide (TBD)
- Security & Compliance Document (TBD)

Document Control

Version	Date	Author	Changes
1.0	2025-11-28	Initial	Initial PRD creation